Loan Default Prediction System

Using Machine Learning Classification Models

Intro to GenAI — Capstone Project Report

Team Members

Name	Role
Dhanvin Vadlamudi	Team Lead
Meka Chaitanya Sai	Team Member
Killi Akshith Kumar	Team Member
Akhil Nath Reddy	Team Member

March 2026

Abstract

Loan defaults are a significant financial risk for lending institutions. In this project, we built an end-to-end machine learning system to predict whether a loan applicant is likely to default. We used a Kaggle dataset of 45,000 loan records with 13 features and trained three classification models: Logistic Regression (baseline), Decision Tree (intermediate), and XGBoost (advanced). After handling class imbalance and tuning hyperparameters, XGBoost achieved the best performance with 91.27% accuracy, 0.82 F1-Score, and 0.98 ROC-AUC. The system was deployed as a Streamlit web application that allows users to input applicant profiles and receive real-time default probability predictions with risk classification.

Contents

1 Introduction **2**

2 Data Description **2**

3 Exploratory Data Analysis (EDA) **3**

4 Data Preprocessing **3**

5 Methodology **4**

5.1 Logistic Regression (Baseline) 4

5.2 Decision Tree (Intermediate) 4

5.3 XGBoost (Best Model) 4

6 Results **5**

7 Optimisation **5**

8 Application — Streamlit Dashboard **6**

9 Team Contribution **8**

10 Conclusion **8**

1 Introduction

Loan defaults cause massive losses for banks. Traditional manual checks are slow and miss patterns in data. Our goal is to build a machine learning system that:

- Predicts the **probability of default** based on applicant profiles.
- Classifies applicants into **Low**, **Medium**, and **High Risk** categories.
- Compares multiple ML models and identifies the best one.
- Deploys the system as a **Streamlit web application** for real-time predictions.

2 Data Description

We used the Loan Approval Classification dataset from Kaggle [2], saved as `loan_data.csv`, containing **45,000 records** with 14 columns.

Table 1: Feature Overview

Feature	**Type**	**Description**
person_age	Numerical	Age of the applicant
person_gender	Categorical	Gender (male/female)
person_education	Categorical	Education level (High School, Associate, Bachelor, Master, Doctorate)
person_income	Numerical	Annual income ($)
person_emp_exp	Numerical	Employment experience (years)
person_home_ownership	Categorical	Home ownership (RENT, OWN, MORTGAGE, OTHER)
loan_amnt	Numerical	Loan amount requested
loan_intent	Categorical	Purpose of the loan (PERSONAL, EDUCATION, MEDICAL, VENTURE, HOMEIMPROVEMENT, DEBTCONSOLIDATION)
loan_int_rate	Numerical	Interest rate (%)
loan_percent_income	Numerical	Loan amount as % of income
cb_person_cred_hist_length	Numerical	Credit history length (years)
credit_score	Numerical	Credit score (300–850)
previous_loan_defaults_on_file	Categorical	Defaulted before (Yes/No)
loan_status	**Target**	**1 = Default, 0 = No Default**

The dataset is **imbalanced**: 35,000 records (78%) are No Default and 10,000 records (22%) are Default.

3 Exploratory Data Analysis (EDA)

We performed thorough Exploratory Data Analysis (EDA) in `preprocessing.ipynb` to understand the underlying distributions and relationships in the dataset prior to modelling. Our analysis included visualising data distributions with histograms and box plots, as well as examining feature interactions through a correlation matrix heatmap.

Missing Values: We identified missing values in numerical columns and opted to drop rows with nulls using `df.dropna()` to ensure data integrity without introducing bias from imputation.

Outliers: We observed significant rightward skewness in several financial features. We capped extreme values at the 99th percentile for `person_age`, `person_income`, and `person_emp_exp` using `df['column'].quantile(0.99)`, standardising the scale for our models.

Key Findings from Correlation Analysis:

- `loan_percent_income` had the highest positive correlation with default, indicating applicants borrowing a large portion of their income are highly risky.
- `previous_loan_defaults_on_file` was strongly correlated — past defaulters have a significantly higher propensity to default again.
- `credit_score` and `person_income` had strong negative correlations — predictably, higher values correspond to lower risk.

4 Data Preprocessing

Robust data preprocessing is essential for training accurate and generalisable models. All preprocessing steps are encapsulated in our `preprocessing.py` script:

1. **Cleaning**: Dropped nulls to maintain pristine data quality, capped outliers at the 99th percentile to prevent models from learning anomalous financial extremities, and binary-encoded gender (male=1, female=0) and previous defaults (Yes=1, No=0).
2. **Ordinal Encoding**: Education mapped to a distinct 0–4 scale (High School < Associate < Bachelor < Master < Doctorate) to preserve the implicit hierarchy of qualifications.
3. **One-Hot Encoding**: Nominal categorical variables such as `person_home_ownership` and `loan_intent` were transformed into sparse matrices via `pd.get_dummies(drop_first=T` avoiding the dummy variable trap.

4. **Train-Test Split**: We partitioned the data using an 80/20 split with `random_state=42`, ensuring a robust test set for realistic evaluation metrics.

5. **Feature Scaling**: `StandardScaler` was applied exclusively for Logistic Regression to converge faster, whereas tree-based models were trained on unscaled data since they rely on value splitting rather than distance metrics.

5 Methodology

We trained three classification models, progressing from simple to advanced:

5.1 Logistic Regression (Baseline)

A linear model using the sigmoid function to output default probability [**?**]. The predicted probability is:

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \tag{1}$$

Configured with `max_iter=1000`, `class_weight='balanced'`, and `StandardScaler`. Chosen as our baseline because it is fast, interpretable, and gives us a reference point.

5.2 Decision Tree (Intermediate)

A non-linear model that splits data at feature thresholds using Gini Impurity [**?**]:

$$Gini = 1 - \sum_{i=1}^{C} p_i^2 \tag{2}$$

where p_i is the proportion of class i in a node. Configured with `max_depth=8`, `min_samples_leaf=10`, and `class_weight='balanced'`. Chosen because it captures feature interactions that logistic regression cannot.

5.3 XGBoost (Best Model)

A gradient boosting algorithm that builds trees sequentially, each minimising the objective function [**?**]:

$$\mathcal{L} = \sum_{i=1}^{n} l(y_i, \hat{y}_i) + \sum_{k=1}^{K} \Omega(f_k) \tag{3}$$

where l is the log-loss function and Ω is the regularisation term that prevents overfitting. Key parameters:

- `n_estimators=200`, `max_depth=8`, `learning_rate=0.05`
- `scale_pos_weight` — auto-calculated ratio to handle class imbalance
- `subsample=0.8`, `colsample_bytree=0.8` — regularisation to prevent overfitting

Chosen because XGBoost is the best-performing algorithm for structured/tabular datasets.

6 Results

All models were evaluated on the **test set** (20% unseen data) using the following metrics:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4}$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \tag{5}$$

ROC-AUC measures the area under the Receiver Operating Characteristic curve — a value of 1.0 means perfect classification.

Table 2: Model Comparison on Test Set

Model	Accuracy (%)	F1-Score	ROC-AUC
Logistic Regression	85.25	0.7389	0.9523
Decision Tree	88.03	0.7714	0.9627
XGBoost	**91.27**	**0.8239**	**0.9763**

XGBoost wins on every metric. The 6% accuracy jump from Logistic Regression to XGBoost shows that boosting captures complex patterns a linear model cannot. The high ROC-AUC (0.98) means the model separates defaulters from non-defaulters almost perfectly.

Top predictive features across all models: `loan_percent_income`, `previous_loan_defaults_on_` `credit_score`, `loan_int_rate`, and `person_income`.

7 Optimisation

Steps taken to improve model performance:

1. **Class Imbalance Handling**: Used `class_weight='balanced'` in Logistic Regression and Decision Tree, and `scale_pos_weight` in XGBoost. Without this, the model would just predict "No Default" for everyone and get 78% accuracy — which is useless.

2. **Hyperparameter Selection**: We tried different values of `max_depth` (5, 8, 10) and found 8 gave the best balance. For XGBoost, `learning_rate=0.05` with 200 trees performed better than the default settings. We also added `min_samples_leaf=10` to prevent the Decision Tree from creating leaves with too few samples.

3. **Outlier Removal**: Capping at the 99th percentile helped models focus on realistic data ranges instead of being skewed by extreme values.

4. **Feature Scaling**: `StandardScaler` improved Logistic Regression convergence since it is sensitive to feature magnitudes.

8 Application — Streamlit Dashboard

We built a web application using Streamlit [**?**] with the following features:

Hosted App: https://genaicapstone-m8ezlrzqqq8ctmmfgsh6ur.streamlit.app/
GitHub Repository: https://github.com/ChaitanyaSai-Meka/gen_ai_capstone

1. **Model Selection**: Users can choose any of the three models from the sidebar.

2. **Model Comparison**: Bar chart (accuracy) and ROC curves displayed side-by-side.

3. **Real-Time Prediction**: Input applicant details via a form and get instant default probability with risk classification (Low/Medium/High).

4. **Feature Importance**: After prediction, shows the top 5 factors affecting the result.

5. **Metrics Dashboard**: Sidebar shows active model's accuracy, F1-Score, and AUC.

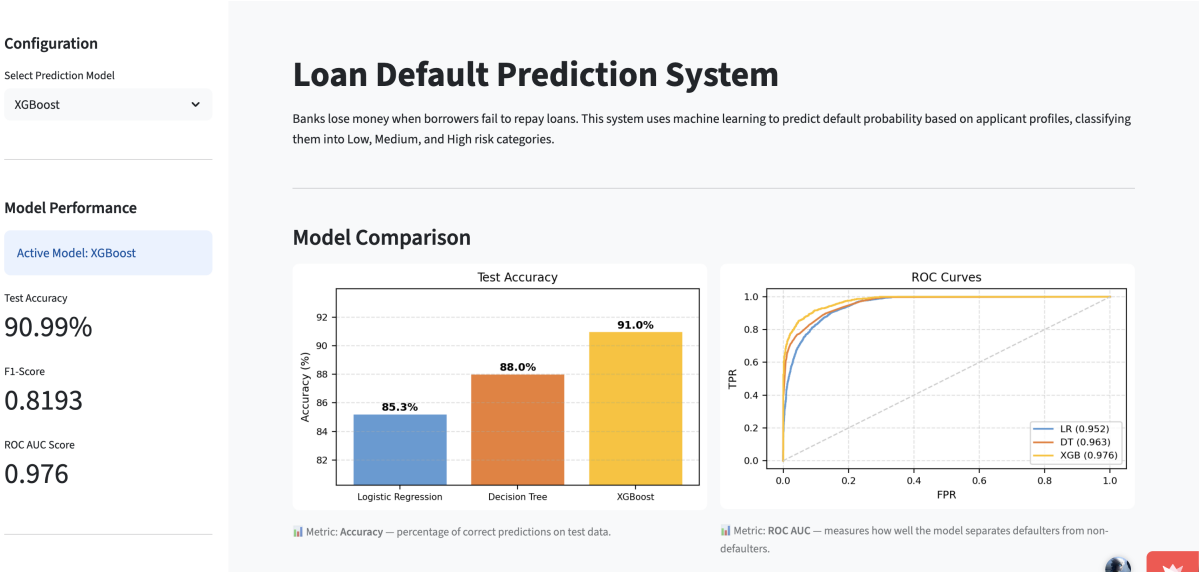

Figure 1: Streamlit Dashboard — Model Comparison with Accuracy Bar Chart and ROC Curves

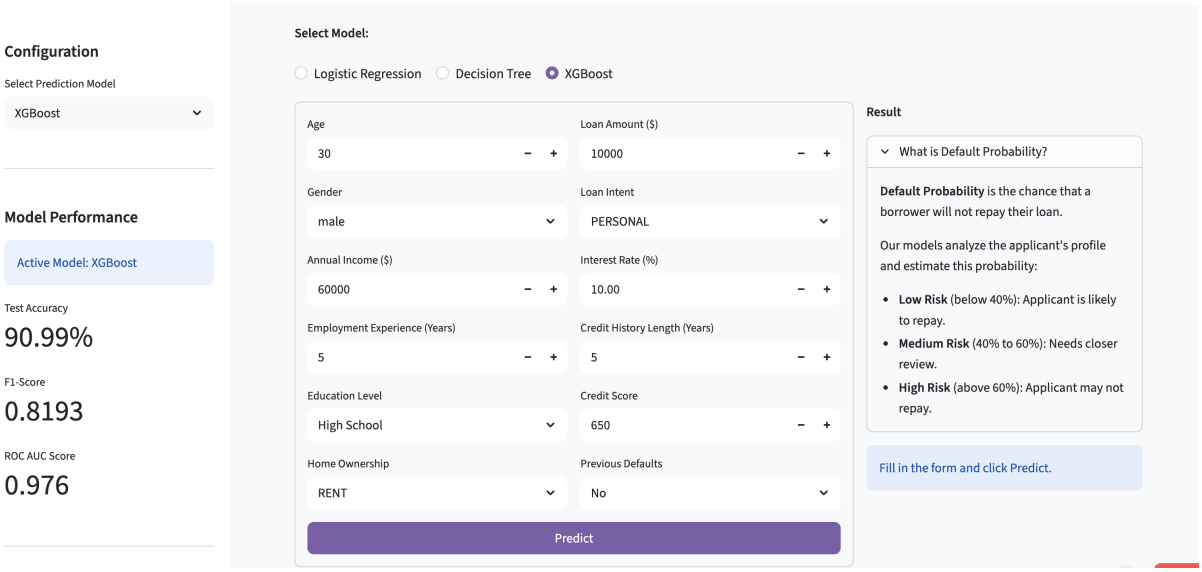

Figure 2: Real-Time Prediction Form with Risk Classification Output

Tech Stack: Streamlit, scikit-learn, XGBoost, Pandas, NumPy, Matplotlib.

9 Team Contribution

Table 3: Team Contribution Breakdown

Team Member	Contribution
Dhanvin Vadlamudi (Team Lead)	Led the overall project architecture, technical direction, and planning. Designed and implemented the XGBoost model, including systematic hyperparameter tuning and performance optimization. Developed the end-to-end preprocessing pipeline. Structured the repository, established coding standards, and coordinated team tasks to ensure timely delivery and smooth integration.
Meka Chaitanya Sai	Built the Decision Tree model. Experimented with different hyperparameters (`max_depth`, `min_samples_leaf`). Assisted with feature encoding.
Killi Akshith Kumar	Built the Logistic Regression model — training, evaluation, and tuning. Performed initial EDA and data exploration in notebooks. Helped with data cleaning logic.
Akhil Nath Reddy	Wrote the project report. Built the Streamlit application (`app.py`). Integrated all models into the dashboard. Assisted with testing and debugging.

10 Conclusion

We built a complete loan default prediction system from scratch. We trained three models (Logistic Regression, Decision Tree, XGBoost), compared their performance, and found that **XGBoost** gave the best results with 91.27% accuracy and 0.98 ROC-AUC.

Key takeaways:

- Handling class imbalance is critical — without it, models just predict the majority class.
- XGBoost outperforms simpler models on tabular data thanks to gradient boosting.
- Deploying as a Streamlit app makes the project practical and demonstrable.
- `loan_percent_income` was consistently the top predictor of default.

References

[1] Streamlit Hosted Application, "Loan Default Prediction Dashboard."
https://genaicapstone-m8ezlrzqqq8ctmmfgsh6ur.streamlit.app/

[2] Kaggle Dataset, "Loan Approval Classification Data."
https://www.kaggle.com/datasets/taweilo/loan-approval-classification-data

[3] GitHub Repository, "GenAI Capstone Source Code."
https://github.com/ChaitanyaSai-Meka/gen_ai_capstone

Academic Integrity Declaration

We, the above-named team members, hereby affirm that the core logic, model architecture, preprocessing pipeline, and Streamlit application code in this project are our own original work. No Generative AI tool was used to directly produce the core implementation. All use of AI tools was limited to research, understanding concepts, and debugging, in accordance with course guidelines.

Team Lead: Dhanvin Vadlamudi
Date: March 2026